

Modeling Computation

13.1 Languages and Grammars

13.2 Finite-State Machines with Output

13.3 Finite-State Machines with No Output

13.4 Language Recognition

13.5 Turing Machines

Computers can perform many tasks. Given a task, two questions arise. The first is: Can it be carried out using a computer? Once we know that this first question has an affirmative answer, we can ask the second question: How can the task be carried out? Models of computation are used to help answer these questions.

We will study three types of structures used in models of computation, namely, grammars, finite-state machines, and Turing machines. Grammars are used to generate the words of a language and to determine whether a word is in a language. Formal languages, which are generated by grammars, provide models both for natural languages, such as English, and for programming languages, such as Pascal, Fortran, Prolog, C, and Java. In particular, grammars are extremely important in the construction and theory of compilers. The grammars that we will discuss were first used by the American linguist Noam Chomsky in the 1950s.

Various types of finite-state machines are used in modeling. All finite-state machines have a set of states, including a starting state, an input alphabet, and a transition function that assigns a next state to every pair of a state and an input. The states of a finite-state machine give it limited memory capabilities. Some finite-state machines produce an output symbol for each transition; these machines can be used to model many kinds of machines, including vending machines, delay machines, binary adders, and language recognizers. We will also study finite-state machines that have no output but do have final states. Such machines are extensively used in language recognition. The strings that are recognized are those that take the starting state to a final state. The concepts of grammars and finite-state machines can be tied together. We will characterize those sets that are recognized by a finite-state machine and show that these are precisely the sets that are generated by a certain type of grammar.

Finally, we will introduce the concept of a Turing machine. We will show how Turing machines can be used to recognize sets. We will also show how Turing machines can be used to compute number-theoretic functions. We will discuss the Church–Turing thesis, which states that every effective computation can be carried out using a Turing machine. We will explain how Turing machines can be used to study the difficulty of solving certain classes of problems. In particular, we will describe how Turing machines are used to classify problems as tractable versus intractable and solvable versus unsolvable.

13.1 Languages and Grammars

13.1.1 Introduction

Words in the English language can be combined in various ways. The grammar of English tells us whether a combination of words is a valid sentence. For instance, *the frog writes neatly* is a valid sentence, because it is formed from a noun phrase, *the frog*, made up of the article *the* and the noun *frog*, followed by a verb phrase, *writes neatly*, made up of the verb *writes* and the adverb *neatly*. We do not care that this is a nonsensical statement, because we are concerned only with the **syntax**, or form, of the sentence, and not its **semantics**, or meaning. We also note that the combination of words *swims quickly mathematics* is not a valid sentence because it does not follow the rules of English grammar.

The syntax of a **natural language**, that is, a spoken language, such as English, French, German, or Spanish, is extremely complicated. In fact, it does not seem possible to specify all the rules of syntax for a natural language. Research in the automatic translation of one language

to another has led to the concept of a **formal language**, which, unlike a natural language, is specified by a well-defined set of rules of syntax. Rules of syntax are important not only in linguistics, the study of natural languages, but also in the study of programming languages.

We will describe the sentences of a formal language using a grammar. The use of grammars helps when we consider the two classes of problems that arise most frequently in applications to programming languages: (1) How can we determine whether a combination of words is a valid sentence in a formal language? (2) How can we generate the valid sentences of a formal language? Before giving a technical definition of a grammar, we will describe an example of a grammar that generates a subset of English. This subset of English is defined using a list of rules that describe how a valid sentence can be produced. We specify that

1. a **sentence** is made up of a **noun phrase** followed by a **verb phrase**;
2. a **noun phrase** is made up of an **article** followed by an **adjective** followed by a **noun**,
or
3. a **noun phrase** is made up of an **article** followed by a **noun**;
4. a **verb phrase** is made up of a **verb** followed by an **adverb**, or
5. a **verb phrase** is made up of a **verb**;
6. an **article** is *a*, or
7. an **article** is *the*;
8. an **adjective** is *large*, or
9. an **adjective** is *hungry*;
10. a **noun** is *rabbit*, or
11. a **noun** is *mathematician*;
12. a **verb** is *eats*, or
13. a **verb** is *hops*;
14. an **adverb** is *quickly*, or
15. an **adverb** is *wildly*.

From these rules we can form valid sentences using a series of replacements until no more rules can be used. For instance, we can follow the sequence of replacements:

```

sentence
noun phrase  verb phrase
article  adjective  noun  verb phrase
article  adjective  noun  verb  adverb
the adjective  noun  verb  adverb
the large noun  verb  adverb
the large rabbit verb  adverb
the large rabbit hops adverb
the large rabbit hops quickly

```

to obtain a valid sentence. It is also easy to see that some other valid sentences are: *a hungry mathematician eats wildly*, *a large mathematician hops*, *the rabbit eats quickly*, and so on. Also, we can see that *the quickly eats mathematician* is not a valid sentence.

13.1.2 Phrase-Structure Grammars

Before we give a formal definition of a grammar, we introduce a little terminology.

Definition 1

A *vocabulary* (or *alphabet*) V is a finite, nonempty set of elements called *symbols*. A *word* (or *sentence*) over V is a string of finite length of elements of V . The *empty string* or *null string*, denoted by λ (and sometimes by ϵ), is the string containing no symbols. The set of all words over V is denoted by V^* . A *language over V* is a subset of V^* .

Note that λ , the empty string, is the string containing no symbols. It is different from $\emptyset$, the empty set. It follows that $\{\lambda\}$ is the set containing exactly one string, namely, the empty string.

Languages can be specified in various ways. One way is to list all the words in the language. Another is to give some criteria that a word must satisfy to be in the language. In this section, we describe another important way to specify a language, namely, through the use of a grammar, such as the set of rules we gave in the introduction to this section. A grammar provides a set of symbols of various types and a set of rules for producing words. More precisely, a grammar has a **vocabulary** V , which is a set of symbols used to derive members of the language. Some of the elements of the vocabulary cannot be replaced by other symbols. These are called **terminals**, and the other members of the vocabulary, which can be replaced by other symbols, are called **nonterminals**. The sets of terminals and nonterminals are usually denoted by T and N , respectively. In the example given in the introduction of the section, the set of terminals is $\{a, the, rabbit, mathematician, hops, eats, quickly, wildly\}$, and the set of nonterminals is $\{\text{sentence, noun phrase, verb phrase, adjective, article, noun, verb, adverb}\}$. There is a special member of the vocabulary called the **start symbol**, denoted by S , which is the element of the vocabulary that we always begin with. In the example in the introduction, the start symbol is **sentence**. The rules that specify when we can replace a string from V^* , the set of all strings of elements in the vocabulary, with another string are called the **productions** of the grammar. We denote by $z_0 \rightarrow z_1$ the production that specifies that z_0 can be replaced by z_1 within a string. The productions in the grammar given in the introduction of this section were listed. The first production, written using this notation, is **sentence** $\rightarrow$ **noun phrase verb phrase**. We summarize this terminology in Definition 2.

The notion of a phrase-structure grammar extends the concept of a rewrite system devised by Axel Thue in the early 20th century.

Definition 2

A *phrase-structure grammar* $G = (V, T, S, P)$ consists of a vocabulary V , a subset T of V consisting of terminal symbols, a start symbol S from V , and a finite set of productions P . The set $V - T$ is denoted by N . Elements of N are called *nonterminal symbols*. Every production in P must contain at least one nonterminal on its left side.

EXAMPLE 1

Let $G = (V, T, S, P)$, where $V = \{a, b, A, B, S\}$, $T = \{a, b\}$, S is the start symbol, and $P = \{S \rightarrow ABa, A \rightarrow BB, B \rightarrow ab, AB \rightarrow b\}$. G is an example of a phrase-structure grammar. ◀

We will be interested in the words that can be generated by the productions of a phrase-structure grammar.

Definition 3

Let $G = (V, T, S, P)$ be a phrase-structure grammar. Let $w_0 = lz_0r$ (that is, the concatenation of l , z_0 , and r) and $w_1 = lz_1r$ be strings over V . If $z_0 \rightarrow z_1$ is a production of G , we say that w_1 is *directly derivable* from w_0 and we write $w_0 \Rightarrow w_1$. If $w_0, w_1, \dots, w_n$ are strings over V such that $w_0 \Rightarrow w_1, w_1 \Rightarrow w_2, \dots, w_{n-1} \Rightarrow w_n$, then we say that w_n is *derivable from w_0* , and we write $w_0 \Rightarrow^* w_n$. The sequence of steps used to obtain w_n from w_0 is called a *derivation*.

EXAMPLE 2 The string $Aaba$ is directly derivable from ABa in the grammar in Example 1 because $B \rightarrow ab$ is a production in the grammar. The string $abababa$ is derivable from ABa because $ABa \Rightarrow Aaba \Rightarrow BBaba \Rightarrow Bababa \Rightarrow abababa$, using the productions $B \rightarrow ab$, $A \rightarrow BB$, $B \rightarrow ab$, and $B \rightarrow ab$ in succession. 

Definition 4 Let $G = (V, T, S, P)$ be a phrase-structure grammar. The *language generated by G* (or the *language of G*), denoted by $L(G)$, is the set of all strings of terminals that are derivable from the starting state S . In other words,

$$L(G) = \{w \in T^* \mid S \xRightarrow{*} w\}.$$

In Examples 3 and 4 we find the language generated by a phrase-structure grammar.

EXAMPLE 3 Let G be the grammar with vocabulary $V = \{S, A, a, b\}$, set of terminals $T = \{a, b\}$, starting symbol S , and productions $P = \{S \rightarrow aA, S \rightarrow b, A \rightarrow aa\}$. What is $L(G)$, the language of this grammar? 

Extra
Examples

Solution: From the start state S we can derive aA using the production $S \rightarrow aA$. We can also use the production $S \rightarrow b$ to derive b . From aA the production $A \rightarrow aa$ can be used to derive aaa . No additional words can be derived. Hence, $L(G) = \{b, aaa\}$. 

EXAMPLE 4 Let G be the grammar with vocabulary $V = \{S, 0, 1\}$, set of terminals $T = \{0, 1\}$, starting symbol S , and productions $P = \{S \rightarrow 11S, S \rightarrow 0\}$. What is $L(G)$, the language of this grammar?

Solution: From S we can derive 0 using $S \rightarrow 0$, or $11S$ using $S \rightarrow 11S$. From $11S$ we can derive either 110 or $1111S$. From $1111S$ we can derive 11110 and $111111S$. At any stage of a derivation we can either add two 1s at the end of the string or terminate the derivation by adding a 0 at the end of the string. We surmise that $L(G) = \{0, 110, 11110, 1111110, \dots\}$, the set of all strings that begin with an even number of 1s and end with a 0. This can be proved using an inductive argument that shows that after n productions have been used, the only strings of terminals generated are those consisting of $n - 1$ concatenations of 11 followed by 0. (This is left as an exercise for the reader.) 

The problem of constructing a grammar that generates a given language often arises. Examples 5, 6, and 7 describe problems of this kind.

EXAMPLE 5 Give a phrase-structure grammar that generates the set $\{0^n 1^n \mid n = 0, 1, 2, \dots\}$.

Solution: Two productions can be used to generate all strings consisting of a string of 0s followed by a string of the same number of 1s, including the null string. The first builds up successively longer strings in the language by adding a 0 at the start of the string and a 1 at the end. The second production replaces S with the empty string. The solution is the grammar $G = (V, T, S, P)$, where $V = \{0, 1, S\}$, $T = \{0, 1\}$, S is the starting symbol, and the productions are

$$\begin{aligned} S &\rightarrow 0S1 \\ S &\rightarrow \lambda. \end{aligned}$$

The verification that this grammar generates the correct set is left as an exercise for the reader. 

Example 5 involved the set of strings made up of 0s followed by 1s, where the number of 0s and 1s are the same. Example 6 considers the set of strings consisting of 0s followed by 1s, where the number of 0s and 1s may differ.

EXAMPLE 6 Find a phrase-structure grammar to generate the set $\{0^m 1^n \mid m \text{ and } n \text{ are nonnegative integers}\}$.

Solution: We will give two grammars G_1 and G_2 that generate this set. This will illustrate that two different grammars can generate the same language.

The grammar G_1 has alphabet $V = \{S, 0, 1\}$; terminals $T = \{0, 1\}$; and productions $S \rightarrow 0S$, $S \rightarrow S1$, and $S \rightarrow \lambda$. Then, G_1 generates the correct set, because using the first production m times puts m 0s at the beginning of the string, and using the second production n times puts n 1s at the end of the string. Furthermore, every string produced by this grammar is of the form $0^m 1^n$, because the only way to add a symbol to the beginning of a string is to add a 0 by applying the production $S \rightarrow 0S$, and the only way to add a symbol to the end of the string is to add a 1 by applying the production $S \rightarrow S1$.

The grammar G_2 has alphabet $V = \{S, A, 0, 1\}$; terminals $T = \{0, 1\}$; and productions $S \rightarrow 0S$, $S \rightarrow 1A$, $S \rightarrow \lambda$, $A \rightarrow 1A$, $A \rightarrow 1$, and $S \rightarrow \lambda$. The details that this grammar generates the correct set are left as an exercise for the reader. 

Sometimes a set that is easy to describe can be generated only by a complicated grammar. Example 7 illustrates this.

EXAMPLE 7 One grammar that generates the set $\{0^n 1^n 2^n \mid n = 0, 1, 2, 3, \dots\}$ is $G = (V, T, S, P)$ with $V = \{0, 1, 2, S, A, B, C\}$; $T = \{0, 1, 2\}$; starting state S ; and productions $S \rightarrow C$, $C \rightarrow 0CAB$, $S \rightarrow \lambda$, $BA \rightarrow AB$, $0A \rightarrow 01$, $1A \rightarrow 11$, $1B \rightarrow 12$, and $2B \rightarrow 22$. We leave it as an exercise for the reader (Exercise 12) to show that this statement is correct. The grammar given is the simplest type of grammar that generates this set, in a sense that will be made clear later in this section. 

13.1.3 Types of Phrase-Structure Grammars

Links  Phrase-structure grammars can be classified according to the types of productions that are allowed. We will describe the classification scheme introduced by Noam Chomsky. In Section 13.4 we will see that the different types of languages defined in this scheme correspond to the classes of languages that can be recognized using different models of computing machines.

A **type 0** grammar has no restrictions on its productions. A **type 1** grammar can have productions of the form $w_1 \rightarrow w_2$, where $w_1 = lAr$ and $w_2 = lwr$, where A is a nonterminal symbol, l and r are strings of zero or more terminal or nonterminal symbols, and w is a nonempty string of terminal or nonterminal symbols. It can also have the production $S \rightarrow \lambda$ as long as S does not appear on the right-hand side of any other production. A **type 2** grammar can have productions only of the form $w_1 \rightarrow w_2$, where w_1 is a single symbol that is not a terminal symbol. A **type 3** grammar can have productions only of the form $w_1 \rightarrow w_2$ with $w_1 = A$ and either $w_2 = aB$ or $w_2 = a$, where A and B are nonterminal symbols and a is a terminal symbol, or with $w_1 = S$ and $w_2 = \lambda$.

Type 2 grammars are called **context-free grammars** because a nonterminal symbol that is the left side of a production can be replaced in a string whenever it occurs, no matter what else is in the string. A language generated by a type 2 grammar is called a **context-free language**. When there is a production of the form $lw_1r \rightarrow lw_2r$ (but not of the form $w_1 \rightarrow w_2$), the grammar is called type 1 or **context-sensitive** because w_1 can be replaced by w_2 only when it is surrounded by the strings l and r . A language generated by a type 1 grammar is called a **context-sensitive language**. Type 3 grammars are also called **regular grammars**. A language generated by a regular grammar is called **regular**. Section 13.4 deals with the relationship between regular languages and finite-state machines.

Of the four types of grammars we have defined, context-sensitive grammars have the most complicated definition. Sometimes, these grammars are defined in a different way. A production of the form $w_1 \rightarrow w_2$ is called **noncontracting** if the length of w_1 is less than or equal to the length of w_2 . According to our characterization of context-sensitive languages, every production in a type 1 grammar, other than the production $S \rightarrow \lambda$, if it is present, is noncontracting. It follows that the lengths of the strings in a derivation in a context-sensitive language are non-decreasing unless the production $S \rightarrow \lambda$ is used. This means that the only way for the empty string to belong to the language generated by a context-sensitive grammar is for the production $S \rightarrow \lambda$ to be part of the grammar. The other way that context-sensitive grammars are defined is by specifying that all productions are noncontracting. A grammar with this property is called **noncontracting** or **monotonic**. The class of noncontracting grammars is not the same as the class of context-sensitive grammars. However, these two classes are closely related; it can be shown that they define the same set of languages except that noncontracting grammars cannot generate any language containing the empty string λ .

EXAMPLE 8 From Example 6 we know that $\{0^m 1^n \mid m, n = 0, 1, 2, \dots\}$ is a regular language, because it can be generated by a regular grammar, namely, the grammar G_2 in Example 6. ◀

**Extra
Examples** ▶

Context-free and regular grammars play an important role in programming languages. Context-free grammars are used to define the syntax of almost all programming languages. These grammars are strong enough to define a wide range of languages. Furthermore, efficient algorithms can be devised to determine whether and how a string can be generated. Regular grammars are used to search text for certain patterns and in lexical analysis, which is the process of transforming an input stream into a stream of tokens for use by a parser.

EXAMPLE 9 It follows from Example 5 that $\{0^n 1^n \mid n = 0, 1, 2, \dots\}$ is a context-free language, because the productions in this grammar are $S \rightarrow 0S1$ and $S \rightarrow \lambda$. However, it is not a regular language. This will be shown in Section 13.4. ◀

EXAMPLE 10 The set $\{0^n 1^n 2^n \mid n = 0, 1, 2, \dots\}$ is a context-sensitive language, because it can be generated by a type 1 grammar, as Example 7 shows, but not by any type 2 language. (This is shown in Exercise 28 in the supplementary exercises at the end of the chapter.) ◀

Table 1 summarizes the terminology used to classify phrase-structure grammars.

13.1.4 Derivation Trees

A derivation in the language generated by a context-free grammar can be represented graphically using an ordered rooted tree, called a **derivation**, or **parse tree**. The root of this tree represents the starting symbol. The internal vertices of the tree represent the nonterminal symbols that

TABLE 1 Types of Grammars.

Type	Restrictions on Productions $w_1 \rightarrow w_2$
0	No restrictions
1	$w_1 = lAr$ and $w_2 = lwr$, where $A \in N$, $l, r, w \in (N \cup T)^*$ and $w \neq \lambda$; or $w_1 = S$ and $w_2 = \lambda$ as long as S is not on the right-hand side of another production
2	$w_1 = A$, where A is a nonterminal symbol
3	$w_1 = A$ and $w_2 = aB$ or $w_2 = a$, where $A \in N$, $B \in N$, and $a \in T$; or $w_1 = S$ and $w_2 = \lambda$

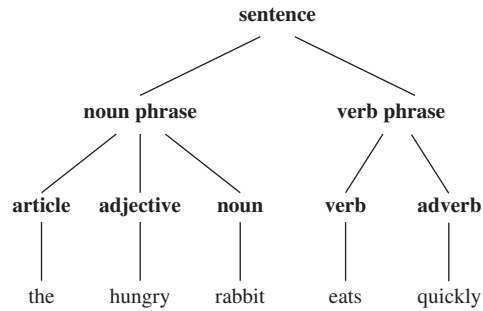


FIGURE 1 A derivation tree.

arise in the derivation. The leaves of the tree represent the terminal symbols that arise. If the production $A \rightarrow w$ arises in the derivation, where w is a word, the vertex that represents A has as children vertices that represent each symbol in w , in order from left to right.

EXAMPLE 11 Construct a derivation tree for the derivation of *the hungry rabbit eats quickly*, given in the introduction of this section.

Solution: The derivation tree is shown in Figure 1. ◀

The problem of determining whether a string is in the language generated by a context-free grammar arises in many applications, such as in the construction of compilers. When given a string, the naive approach for determining whether it is in the language generated by a grammar is to look for a sequence of productions that can be applied, beginning at the start state, that lead to the given string. When following such an approach, it is useful to think a few moves ahead. This approach is known as **top-down parsing**. A second approach, known as **bottom-up parsing**, is to work backward from the given string with the goal of undoing productions one-by-one to reach the start symbol. We illustrate these two approaches to this problem in Example 12.

EXAMPLE 12 Determine whether the word *cbab* belongs to the language generated by the grammar $G = (V, T, S, P)$, where $V = \{a, b, c, A, B, C, S\}$, $T = \{a, b, c\}$, S is the starting symbol, and the productions are

$$\begin{aligned}
 S &\rightarrow AB \\
 A &\rightarrow Ca \\
 B &\rightarrow Ba \\
 B &\rightarrow Cb \\
 B &\rightarrow b \\
 C &\rightarrow cb \\
 C &\rightarrow b.
 \end{aligned}$$

Solution: A top-down approach to this problem is to begin with S and attempt to derive *cbab* using a series of productions. Because there is only one production with S on its left-hand side, we must start with $S \Rightarrow AB$. Next we use the only production that has A on its left-hand side,

Links



©SASCHA SCHUERMANN/
AFP/Getty Images

AVRAM NOAM CHOMSKY (BORN 1928) Noam Chomsky, born in Philadelphia, is the son of a Hebrew scholar. He received his B.A., M.A., and Ph.D. in linguistics, all from the University of Pennsylvania. He was on the staff of the University of Pennsylvania from 1950 until 1951. In 1955 he joined the faculty at M.I.T., beginning his M.I.T. career teaching engineers French and German. Chomsky is currently the Ferrari P. Ward Professor of foreign languages and linguistics at M.I.T. He is known for his many fundamental contributions to linguistics, including the study of grammars. Chomsky is also widely known for his outspoken political activism.

namely, $A \rightarrow Ca$, to obtain $S \Rightarrow AB \Rightarrow CaB$. Because $cbab$ begins with the symbols cb , we use the production $C \rightarrow cb$. This gives us $S \Rightarrow AB \Rightarrow CaB \Rightarrow cbaB$. We finish by using the production $B \rightarrow b$, to obtain $S \Rightarrow AB \Rightarrow CaB \Rightarrow cbaB \Rightarrow cbab$.

We will solve the same problem using bottom-up parsing. Because $cbab$ is the string to be derived, we can use the production $C \rightarrow cb$, so that $Cab \Rightarrow cbab$. Then, we can use the production $A \rightarrow Ca$, so that $Ab \Rightarrow Cab \Rightarrow cbab$. Using the production $B \rightarrow b$ gives $AB \Rightarrow Ab \Rightarrow Cab \Rightarrow cbab$. Finally, using $S \rightarrow AB$ shows that a complete derivation for $cbab$ is $S \Rightarrow AB \Rightarrow Ab \Rightarrow Cab \Rightarrow cbab$. ▶

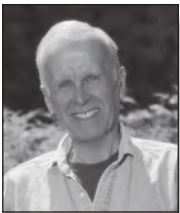
Example 12 presents the solution of a parsing problem using both top-down and bottom-up parsing. Both approaches were easy to use for that problem. However, parsing problems can be quite challenging. That is, it can be quite challenging to determine whether a string is in the language generated by a context-free grammar. Because parsing is so important, many strategies and algorithms have been devised for top-down and for bottom-up parsing. These algorithms are beyond the scope of this book. The interested reader should consult [AhLaSeUI06] to learn more.

13.1.5 Backus–Naur Form

Links ▶

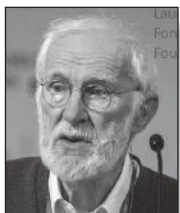
There is another notation that is sometimes used to specify a type 2 grammar, called the **Backus–Naur form (BNF)**, after John Backus, who invented it, and Peter Naur, who refined

Links ▶



Courtesy of Louis Bachrach

JOHN BACKUS (1924–2007) John Backus was born in Philadelphia and grew up in Wilmington, Delaware. He attended the Hill School in Pottstown, Pennsylvania. He needed to attend summer school every year because he disliked studying and was not a serious student. But he enjoyed spending his summers in New Hampshire where he attended summer school and amused himself with summer activities, including sailing. He obliged his father by enrolling at the University of Virginia to study chemistry. But he quickly decided chemistry was not for him, and in 1943 he entered the army, where he received medical training and worked in a neuro-surgery ward in an army hospital. Ironically, Backus was soon diagnosed with a bone tumor in his skull and was fitted with a metal plate. His medical work in the army convinced him to try medical school, but he abandoned this after nine months because he disliked the rote memorization required. After dropping out of medical school, he entered a school for radio technicians because he wanted to build his own high fidelity set. A teacher in this school recognized his potential and asked him to help with some mathematical calculations needed for an article in a magazine. Finally, Backus found what he was interested in: mathematics and its applications. He enrolled at Columbia University, from which he received both bachelor's and master's degrees in mathematics. Backus joined IBM as a programmer in 1950. He participated in the design and development of two of IBM's early computers. From 1954 to 1958 he led the IBM group that developed FORTRAN. Backus became a staff member at the IBM Watson Research Center in 1958. He was part of the committees that designed the programming language ALGOL using what is now called the Backus–Naur form for the description of the syntax of this language. (The development of ALGOL led to the development of other programming languages, including Pascal and C.) Later, Backus worked on the mathematics of families of sets and on a functional style of programming. Backus became an IBM Fellow in 1963, and he received the National Medal of Science in 1974 and the prestigious Turing Award from the Association for Computing Machinery in 1977.



©Heidelberg Laureate Forum Foundation

PETER NAUR (1928–2016) Peter Naur was born in Frederiksberg, near Copenhagen. As a boy he became interested in astronomy. Not only did he observe heavenly bodies, but he also computed the orbits of comets and asteroids. Naur attended Copenhagen University, receiving his degree in 1949. He spent 1950 and 1951 in Cambridge, where he used an early computer to calculate the motions of comets and planets. After returning to Denmark, he continued working in astronomy but kept his ties to computing. In 1955 he served as a consultant to the building of the first Danish computer. In 1959 Naur made the switch from astronomy to computing as a full-time activity. His first job as a full-time computer scientist was participating in the development of the programming language ALGOL. From 1960 to 1967 he worked on the development of compilers for ALGOL and COBOL. From 1969 until 1999 he was professor of computer science at Copenhagen University, where he has worked in the area of programming methodology. His research interests included the design, structure, and performance of computer programs. Naur was a pioneer in both the areas of software architecture and software engineering. He rejected the view that computer programming is a branch of mathematics and preferred that computer science be called *datalogy*.

The ancient Indian grammarian Pāṇini specified Sanskrit using 3959 rules; Backus–Naur form is sometimes called Backus–Pāṇini form.

it for use in the specification of the programming language ALGOL. (Surprisingly, a notation quite similar to the Backus–Naur form was used approximately 2500 years ago to describe the grammar of Sanskrit.) The Backus–Naur form is used to specify the syntactic rules of many computer languages, including Java. The productions in a type 2 grammar have a single nonterminal symbol as their left-hand side. Instead of listing all the productions separately, we can combine all those with the same nonterminal symbol on the left-hand side into one statement. Instead of using the symbol $\rightarrow$ in a production, we use the symbol $::=$. We enclose all nonterminal symbols in brackets, $\langle \rangle$, and we list all the right-hand sides of productions in the same statement, separating them by bars. For instance, the productions $A \rightarrow Aa$, $A \rightarrow a$, and $A \rightarrow AB$ can be combined into $\langle A \rangle ::= \langle A \rangle a \mid a \mid \langle A \rangle \langle B \rangle$.

Example 13 illustrates how the Backus–Naur form is used to describe the syntax of programming languages. Our example comes from the original use of Backus–Naur form in the description of ALGOL 60.

EXAMPLE 13

Extra
Examples

In ALGOL 60 an identifier (which is the name of an entity such as a variable) consists of a string of alphanumeric characters (that is, letters and digits) and must begin with a letter. We can use these rules in Backus–Naur to describe the set of allowable identifiers:

$$\begin{aligned}\langle \text{identifier} \rangle &::= \langle \text{letter} \rangle \mid \langle \text{identifier} \rangle \langle \text{letter} \rangle \mid \langle \text{identifier} \rangle \langle \text{digit} \rangle \\ \langle \text{letter} \rangle &::= a \mid b \mid \cdots \mid y \mid z \quad \text{the ellipsis indicates that all 26 letters are included} \\ \langle \text{digit} \rangle &::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9\end{aligned}$$

For example, we can produce the valid identifier $x99a$ by using the first rule to replace $\langle \text{identifier} \rangle$ by $\langle \text{identifier} \rangle \langle \text{letter} \rangle$, the second rule to obtain $\langle \text{identifier} \rangle a$, the first rule twice to obtain $\langle \text{identifier} \rangle \langle \text{digit} \rangle \langle \text{digit} \rangle a$, the third rule twice to obtain $\langle \text{identifier} \rangle 99a$, the first rule to obtain $\langle \text{letter} \rangle 99a$, and finally the second rule to obtain $x99a$. 

EXAMPLE 14

What is the Backus–Naur form of the grammar for the subset of English described in the introduction to this section?

Solution: The Backus–Naur form of this grammar is

$$\begin{aligned}\langle \text{sentence} \rangle &::= \langle \text{noun phrase} \rangle \langle \text{verb phrase} \rangle \\ \langle \text{noun phrase} \rangle &::= \langle \text{article} \rangle \langle \text{adjective} \rangle \langle \text{noun} \rangle \mid \langle \text{article} \rangle \langle \text{noun} \rangle \\ \langle \text{verb phrase} \rangle &::= \langle \text{verb} \rangle \langle \text{adverb} \rangle \mid \langle \text{verb} \rangle \\ \langle \text{article} \rangle &::= a \mid the \\ \langle \text{adjective} \rangle &::= large \mid hungry \\ \langle \text{noun} \rangle &::= rabbit \mid mathematician \\ \langle \text{verb} \rangle &::= eats \mid hops \\ \langle \text{adverb} \rangle &::= quickly \mid wildly\end{aligned}$$


EXAMPLE 15

Give the Backus–Naur form for the production of signed integers in decimal notation. (A **signed integer** is a nonnegative integer preceded by a plus sign or a minus sign.)

Solution: The Backus–Naur form for a grammar that produces signed integers is

$$\begin{aligned}\langle \text{signed integer} \rangle &::= \langle \text{sign} \rangle \langle \text{integer} \rangle \\ \langle \text{sign} \rangle &::= + \mid - \\ \langle \text{integer} \rangle &::= \langle \text{digit} \rangle \mid \langle \text{digit} \rangle \langle \text{integer} \rangle \\ \langle \text{digit} \rangle &::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9\end{aligned}$$


The Backus–Naur form, with a variety of extensions, is used extensively to specify the syntax of programming languages, such as Java and LISP; database languages, such as SQL; and markup languages, such as XML. Some extensions of the Backus–Naur form that are commonly used in the description of programming languages are introduced in the preamble to Exercise 34.

Exercises

Exercises 1–3 refer to the grammar with start symbol **sentence**, set of terminals $T = \{the, sleepy, happy, tortoise, hare, passes, runs, quickly, slowly\}$, set of nonterminals $N = \{\text{noun phrase, transitive verb phrase, intransitive verb phrase, article, adjective, noun, verb, adverb}\}$, and productions:

sentence $\rightarrow$ **noun phrase** **transitive verb phrase**
 noun phrase
sentence $\rightarrow$ **noun phrase** **intransitive verb phrase**
noun phrase $\rightarrow$ **article** **adjective** **noun**
noun phrase $\rightarrow$ **article** **noun**
transitive verb phrase $\rightarrow$ **transitive verb**
intransitive verb phrase $\rightarrow$ **intransitive verb** **adverb**
intransitive verb phrase $\rightarrow$ **intransitive verb**
article $\rightarrow$ *the*

adjective $\rightarrow$ *sleepy*
adjective $\rightarrow$ *happy*
noun $\rightarrow$ *tortoise*
noun $\rightarrow$ *hare*
transitive verb $\rightarrow$ *passes*
intransitive verb $\rightarrow$ *runs*
adverb $\rightarrow$ *quickly*
adverb $\rightarrow$ *slowly*

- Use the set of productions to show that each of these sentences is a valid sentence.
 - the happy hare runs*
 - the sleepy tortoise runs quickly*
 - the tortoise passes the hare*
 - the sleepy hare passes the happy tortoise*
- Find five other valid sentences, besides those given in Exercise 1.
- Show that *the hare runs the sleepy tortoise* is not a valid sentence.
- Let $G = (V, T, S, P)$ be the phrase-structure grammar with $V = \{0, 1, A, S\}$, $T = \{0, 1\}$, and set of productions P consisting of $S \rightarrow 1S$, $S \rightarrow 00A$, $A \rightarrow 0A$, and $A \rightarrow 0$.
 - Show that 111000 belongs to the language generated by G .
 - Show that 11001 does not belong to the language generated by G .
 - What is the language generated by G ?

- Let $G = (V, T, S, P)$ be the phrase-structure grammar with $V = \{0, 1, A, B, S\}$, $T = \{0, 1\}$, and set of productions P consisting of $S \rightarrow 0A$, $S \rightarrow 1A$, $A \rightarrow 0B$, $B \rightarrow 1A$, $B \rightarrow 1$.
 - Show that 10101 belongs to the language generated by G .
 - Show that 10110 does not belong to the language generated by G .
 - What is the language generated by G ?
- * Let $V = \{S, A, B, a, b\}$ and $T = \{a, b\}$. Find the language generated by the grammar (V, T, S, P) when the set P of productions consists of
 - $S \rightarrow AB, A \rightarrow ab, B \rightarrow bb$.
 - $S \rightarrow AB, S \rightarrow aA, A \rightarrow a, B \rightarrow ba$.
 - $S \rightarrow AB, S \rightarrow AA, A \rightarrow aB, A \rightarrow ab, B \rightarrow b$.
 - $S \rightarrow AA, S \rightarrow B, A \rightarrow aaA, A \rightarrow aa, B \rightarrow bB, B \rightarrow b$.
 - $S \rightarrow AB, A \rightarrow aAb, B \rightarrow bBa, A \rightarrow \lambda, B \rightarrow \lambda$.
- Construct a derivation of 0^31^3 using the grammar given in Example 5.
- Show that the grammar given in Example 5 generates the set $\{0^n1^n \mid n = 0, 1, 2, \dots\}$.
- Construct a derivation of 0^21^4 using the grammar G_1 in Example 6.
 - Construct a derivation of 0^21^4 using the grammar G_2 in Example 6.
- Show that the grammar G_1 given in Example 6 generates the set $\{0^m1^n \mid m, n = 0, 1, 2, \dots\}$.
 - Show that the grammar G_2 in Example 6 generates the same set.
- Construct a derivation of $0^21^22^2$ in the grammar given in Example 7.
- * Show that the grammar given in Example 7 generates the set $\{0^n1^n2^n \mid n = 0, 1, 2, \dots\}$.
- Find a phrase-structure grammar for each of these languages.
 - the set consisting of the bit strings 0, 1, and 11
 - the set of bit strings containing only 1s
 - the set of bit strings that start with 0 and end with 1
 - the set of bit strings that consist of a 0 followed by an even number of 1s
- Find a phrase-structure grammar for each of these languages.
 - the set consisting of the bit strings 10, 01, and 101
 - the set of bit strings that start with 00 and end with one or more 1s
 - the set of bit strings consisting of an even number of 1s followed by a final 0

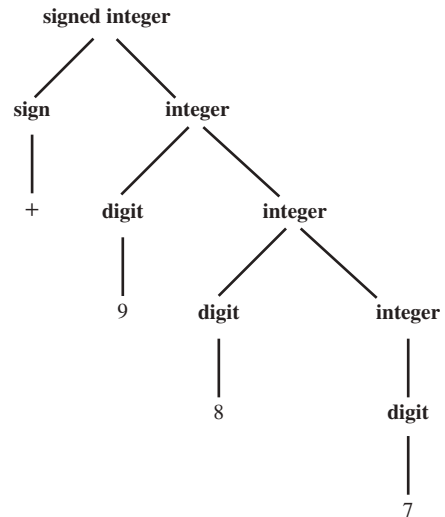
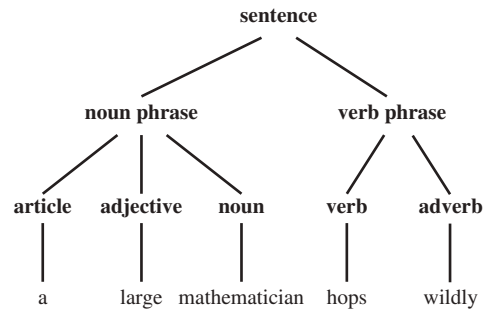
- d) the set of bit strings that have neither two consecutive 0s nor two consecutive 1s
- *15. Find a phrase-structure grammar for each of these languages.
- the set of all bit strings containing an even number of 0s and no 1s
 - the set of all bit strings made up of a 1 followed by an odd number of 0s
 - the set of all bit strings containing an even number of 0s and an even number of 1s
 - the set of all strings containing 10 or more 0s and no 1s
 - the set of all strings containing more 0s than 1s
 - the set of all strings containing an equal number of 0s and 1s
 - the set of all strings containing an unequal number of 0s and 1s
16. Construct phrase-structure grammars to generate each of these sets.
- $\{1^n \mid n \geq 0\}$
 - $\{10^n \mid n \geq 0\}$
 - $\{(11)^n \mid n \geq 0\}$
17. Construct phrase-structure grammars to generate each of these sets.
- $\{0^n \mid n \geq 0\}$
 - $\{1^n 0 \mid n \geq 0\}$
 - $\{(000)^n \mid n \geq 0\}$
18. Construct phrase-structure grammars to generate each of these sets.
- $\{01^{2n} \mid n \geq 0\}$
 - $\{0^n 1^{2n} \mid n \geq 0\}$
 - $\{0^n 1^m 0^n \mid m \geq 0 \text{ and } n \geq 0\}$
19. Let $V = \{S, A, B, a, b\}$ and $T = \{a, b\}$. Determine whether $G = (V, T, S, P)$ is a type 0 grammar but not a type 1 grammar, a type 1 grammar but not a type 2 grammar, or a type 2 grammar but not a type 3 grammar if P , the set of productions, is
- $S \rightarrow aAB, A \rightarrow Bb, B \rightarrow \lambda$.
 - $S \rightarrow aA, A \rightarrow a, A \rightarrow b$.
 - $S \rightarrow ABa, AB \rightarrow a$.
 - $S \rightarrow ABA, A \rightarrow aB, B \rightarrow ab$.
 - $S \rightarrow bA, A \rightarrow B, B \rightarrow a$.
 - $S \rightarrow aA, aA \rightarrow B, B \rightarrow aA, A \rightarrow b$.
 - $S \rightarrow bA, A \rightarrow b, S \rightarrow \lambda$.
 - $S \rightarrow AB, B \rightarrow aAb, aAb \rightarrow b$.
 - $S \rightarrow aA, A \rightarrow bB, B \rightarrow b, B \rightarrow \lambda$.
 - $S \rightarrow A, A \rightarrow B, B \rightarrow \lambda$.

20. A **palindrome** is a string that reads the same backward as it does forward, that is, a string w , where $w = w^R$, where w^R is the reversal of the string w . Find a context-free grammar that generates the set of all palindromes over the alphabet $\{0, 1\}$.

- *21. Let G_1 and G_2 be context-free grammars, generating the languages $L(G_1)$ and $L(G_2)$, respectively. Show that there is a context-free grammar generating each of these sets.

- $L(G_1) \cup L(G_2)$
- $L(G_1)L(G_2)$
- $L(G_1)^*$

22. Find the strings constructed using the derivation trees shown here.



23. Construct derivation trees for the sentences in Exercise 1.

24. Let G be the grammar with $V = \{a, b, c, S\}$; $T = \{a, b, c\}$; starting symbol S ; and productions $S \rightarrow abS$, $S \rightarrow bcS$, $S \rightarrow bbS$, $S \rightarrow a$, and $S \rightarrow cb$. Construct derivation trees for

- $bcbbba$.
- $bbbcbbba$.
- $bcabbbbbbcb$.

- *25. Use top-down parsing to determine whether each of the following strings belongs to the language generated by the grammar in Example 12.

- $baba$
- $abab$
- $cbaba$
- $bbbcba$

- *26. Use bottom-up parsing to determine whether the strings in Exercise 25 belong to the language generated by the grammar in Example 12.

27. Construct a derivation tree for -109 using the grammar given in Example 15.
28. a) Explain what the productions are in a grammar if the Backus–Naur form for productions is as follows:

$$\begin{aligned}\langle expression \rangle &::= (\langle expression \rangle) \mid \\ &\quad \langle expression \rangle + \langle expression \rangle \mid \\ &\quad \langle expression \rangle * \langle expression \rangle \mid \\ &\quad \langle variable \rangle \\ \langle variable \rangle &::= x \mid y\end{aligned}$$

- b) Find a derivation tree for $(x * y) + x$ in this grammar.
29. a) Construct a phrase-structure grammar that generates all signed decimal numbers, consisting of a sign, either $+$ or $-$; a nonnegative integer; and a decimal fraction that is either the empty string or a decimal point followed by a positive integer, where initial zeros in an integer are allowed.
- b) Give the Backus–Naur form of this grammar.
- c) Construct a derivation tree for -31.4 in this grammar.
30. a) Construct a phrase-structure grammar for the set of all fractions of the form a/b , where a is a signed integer in decimal notation and b is a positive integer.
- b) What is the Backus–Naur form for this grammar?
- c) Construct a derivation tree for $+311/17$ in this grammar.
31. Give production rules in Backus–Naur form for an identifier if it can consist of
- one or more lowercase letters.
 - at least three but no more than six lowercase letters.
 - one to six uppercase or lowercase letters beginning with an uppercase letter.
 - a lowercase letter, followed by a digit or an underscore, followed by three or four alphanumeric characters (lower or uppercase letters and digits).
32. Give production rules in Backus–Naur form for the name of a person if this name consists of a first name, which is a string of letters, where only the first letter is uppercase; a middle initial; and a last name, which can be any string of letters.
33. Give production rules in Backus–Naur form that generate all identifiers in the C programming language. In C an identifier starts with a letter or an underscore ($_$) that is followed by one or more lowercase letters, uppercase letters, underscores, and digits.

➤ Several extensions to Backus–Naur form are commonly used to define phrase-structure grammars. In one such extension, a question mark (?) indicates that the symbol, or group of symbols inside parentheses, to its left can appear zero or once (that

is, it is optional), an asterisk (*) indicates that the symbol to its left can appear zero or more times, and a plus (+) indicates that the symbol to its left can appear one or more times. These extensions are part of **extended Backus–Naur form (EBNF)**, and the symbols $?$, $*$, and $+$ are called **metacharacters**. In EBNF the brackets used to denote nonterminals are usually not shown.

34. Describe the set of strings defined by each of these sets of productions in EBNF.
- $string ::= L+D?L+$
 $L ::= a \mid b \mid c$
 $D ::= 0 \mid 1$
 - $string ::= sign D+ \mid D+$
 $sign ::= + \mid -$
 $D ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$
 - $string ::= L*(D+)?L*$
 $L ::= x \mid y$
 $D ::= 0 \mid 1$
35. Give production rules in extended Backus–Naur form that generate all decimal numerals consisting of an optional sign, a nonnegative integer, and a decimal fraction that is either the empty string or a decimal point followed by an optional positive integer optionally preceded by some number of zeros.
36. Give production rules in extended Backus–Naur form that generate a sandwich if a sandwich consists of a lower slice of bread; mustard or mayonnaise; optional lettuce; an optional slice of tomato; one or more slices of either turkey, chicken, or roast beef (in any combination); optionally some number of slices of cheese; and a top slice of bread.
37. Give production rules in extended Backus–Naur form for identifiers in the C programming language (see Exercise 33).
38. Describe how productions for a grammar in extended Backus–Naur form can be translated into a set of productions for the grammar in Backus–Naur form.

This is the Backus–Naur form that describes the syntax of expressions in postfix (or reverse Polish) notation.

$$\begin{aligned}\langle expression \rangle &::= \langle term \rangle \mid \langle term \rangle \langle term \rangle \langle addOperator \rangle \\ \langle addOperator \rangle &::= + \mid - \\ \langle term \rangle &::= \langle factor \rangle \mid \langle factor \rangle \langle factor \rangle \langle mulOperator \rangle \\ \langle mulOperator \rangle &::= * \mid / \\ \langle factor \rangle &::= \langle identifier \rangle \mid \langle expression \rangle \\ \langle identifier \rangle &::= a \mid b \mid \cdots \mid z\end{aligned}$$

39. For each of these strings, determine whether it is generated by the grammar given for postfix notation. If it is, find the steps used to generate the string
- $abc*+$
 - $xy++$
 - $xy-z*$
 - $wxyz-* /$
 - $ade-*$